

EEE 312 (January 2026)
Digital Signal Processing I Laboratory

Final Project Report

Section: C2 Group: 04

Audio-Visual Speaker Isolation
Click a face in a video, hear only that person

Course Instructors:

[Name of course instructor 1], [Designation]
[Name of course instructor 2], [Designation]

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____
Full Name: Redwan Nur Alam Shilad
Student ID: 2206177

Signature: _____
Full Name: Ahnaf Hasan Adeeb
Student ID: 2206178

Signature: _____
Full Name: S. M. Khairul Islam Rifat
Student ID: 2206179

Signature: _____
Full Name: Md. Shahriare Arefin Shuvo
Student ID: 2206180

Table of Contents

1	Abstract.....	1
2	Introduction	1
3	Design	3
3.1	Problem Formulation (PO(b))	3
3.1.1	Identification of Scope	3
3.1.2	Literature Review.....	4
3.1.3	Formulation of Problem	5
3.1.4	Analysis.....	6
3.2	Design Method (PO(a))	7
3.2.1	Pre-processing and sampling-rate conversion.....	7
3.2.2	Face tracking, re-identification and mouth regions.....	7
3.2.3	Lip-conditioned extraction	8
3.2.4	Refinement: subtracting the other speakers, not yourself	9
3.2.5	Coherent leak cancellation (strict isolation).....	9
3.2.6	Word-safe silence gate	10
3.2.7	Interactive player.....	11
3.2.8	Software architecture	12
3.3	Simulation Model	13
3.4	Source Code of the Core Signal-Processing Routines.....	14
4	Implementation	15
4.1	Description	15
5	Design Analysis and Evaluation	19
5.1	Novelty	19
5.2	Design Considerations (PO(c)).....	20
5.2.1	Considerations to public health and safety.....	20
5.2.2	Considerations to environment	20
5.2.3	Considerations to cultural and societal needs	20
5.3	Investigations (PO(d))	21
5.3.1	Design of Experiment	21
5.3.2	Data Collection	22
5.3.3	Results and Analysis	22
5.3.4	Interpretation and Conclusions on Data	24

5.4	Limitations of Tools (PO(e))	25
5.5	Impact Assessment (PO(f))	25
5.5.1	Assessment of Societal and Cultural Issues	25
5.5.2	Assessment of Health and Safety Issues	26
5.5.3	Assessment of Legal Issues.....	26
5.6	Sustainability Evaluation (PO(g))	26
5.7	Ethical Issues (PO(h)).....	27
6	Reflection on Individual and Team work (PO(i))	28
6.1	Individual Contribution of Each Member	28
6.2	Mode of TeamWork	28
6.3	Diversity Statement of Team.....	28
6.4	Log Book of Project Implementation.....	28
7	Communication to External Stakeholders (PO(j))	29
7.1	Executive Summary.....	29
7.2	User Manual	29
7.3	Github Link	30
7.4	YouTube Link	31
8	Project Management and Cost Analysis (PO(k)).....	31
8.1	Bill of Materials.....	31
8.2	Calculation of Per Unit Cost of Prototype.....	31
8.3	Calculation of Per Unit Cost of Mass-Produced Unit	31
8.4	Timeline of Project Implementation.....	32
9	Future Work (PO(l)).....	32
10	References	32

1 Abstract

When several people talk at once, a single microphone records the sum of their voices, and a listener cannot follow one of them. We built a system that uses the video as well as the audio: the user uploads a video, clicks a face, and hears only that person, with instant switching between speakers. For every visible face, a lip-conditioned network (AV-MossFormer2 target-speaker extraction) estimates that person's voice from the 16 kHz mixture and a 112×112 video of the mouth. Around this model we designed the signal-processing chain: frame-aligned 2 s decoding windows whose edges are discarded; a refinement pass that re-extracts each voice from the mixture minus the other speakers while protecting the part that is phase-coherent with the target; a coherent leak canceller that removes, bin by bin, the part of one channel that is a copy of another; and a Schmitt-trigger gate that produces exact digital silence in pauses without clipping words. A browser player built on the Web Audio API plays every channel from one buffer on one clock, so switching is a 25 ms click-free crossfade. On our own recording of two people reading different texts at the same time, the word error rate of the isolated channels fell from 110–128 % with audio-only separation to 3.5–5.4 %, and on a ground-truth leak bench the signal-to-interference ratio rose from 9.9/6.9 dB to 16.3/13.4 dB. The system runs locally at about twice real time on an 8 GB laptop GPU. Its measured limits are the 8 kHz audio bandwidth, the need for a visible face of about 150 pixels, and 13–16 dB of residual cross-talk when two similar voices overlap without pause.

Keywords: audio-visual speech separation, target-speaker extraction, cocktail-party problem, short-time Fourier transform, Schmitt trigger, Web Audio API.

2 Introduction

In a room where several people talk at the same time, a listener with normal hearing can still follow one of them. This is the cocktail-party effect described by Cherry [1]: the brain combines the two ears, knowledge of the voice and, strongly, the sight of the speaker's lips. A recording loses most of these cues. A phone, laptop or television microphone captures a single channel in which the voices are simply added,

$$x(t) = \sum_{k=1}^K s_k(t) + n(t) \quad (1)$$

so every region of the time–frequency plane may hold energy from more than one person. Listeners who find it hard to separate voices, including many hearing-impaired and non-native listeners, struggle with such recordings, and the same difficulty appears whenever a meeting, interview, class or televised debate has to be transcribed or reviewed.

This project builds a tool for that situation. The user uploads a video, clicks a face and hears only that person; clicking another face switches to the other voice instantly, without interrupting playback. Figure 1 shows our showcase recording, in which two of us read two different texts at the same time into one phone. It is a deliberately hard test: the voices are similar and overlap continuously.



Figure 1: Our showcase recording. Two team members read different texts at the same time into one iPhone microphone; Speaker A (right, black shirt) and Speaker B (left, white shirt) must be separated from a single audio channel.

The task is a complex engineering problem: it cannot be solved by applying a standard procedure, and it needs in-depth knowledge from several fields that interact. Table 1 maps it to the attributes of a complex engineering problem addressed by this course.

Attribute	How it appears in this project
P1: Depth of knowledge required	Discrete-time signal processing (sampling-rate conversion, STFT analysis and synthesis, least-squares regression in the time–frequency domain, hysteresis detection), deep-learning inference (transformer separators, lip-reading front-ends), computer vision (face detection, tracking and re-identification) and real-time web audio all have to work together.
P2: Range of conflicting requirements	Stronger suppression of the other voice also removes parts of the chosen voice; a gate that produces more silence clips more quiet words; longer decoding windows give the network more context but made the two outputs more alike; local processing protects privacy but limits computing power to one laptop GPU.
P3: Depth of analysis required	There is no obvious solution. Our first design (blind separation followed by lip–audio matching) turned out to perform at chance, and the failure could only be found by building diagnostic measurements and a ground-truth bench.
P6: Extent of stakeholder involvement	Listeners who need clearer speech, the people who are recorded (consent and privacy), the owners of broadcast content and the licensors of the pretrained models have different and partly conflicting interests.
P7: Interdependence (also present)	Eight processing stages interact: for example, the right gate thresholds depend on how much leakage the separator leaves, and the separator’s quality depends on the face crop.

Table 1: Complex-engineering-problem attributes of the project.

Several families of solutions were possible (Table 2); Section 3.1.2 reviews them in detail.

Approach	What it needs	Fit to this problem
Microphone-array beamforming or independent component analysis	several microphones	Not applicable: our recordings have one channel, so the problem is underdetermined.
Audio-only blind separation (e.g. SepFormer)	one microphone	Does not know which voice belongs to which face; fixed number of outputs; on our two similar voices both texts appeared in both outputs.
Blind separation followed by lip-audio matching (our V2)	one microphone and the video	The matching step was measured at chance on a real talk show.
Extraction with a voice enrolment (e.g. VoiceFilter)	a clean sample of each person's voice	Clean samples are rarely available for an arbitrary video.
Audio-visual target-speaker extraction (chosen)	one microphone and a visible face	The lips select the voice: one extraction per face, any number of faces, no enrolment.

Table 2: Alternative solutions considered.

The contributions of this project are:

- a complete pipeline from an uploaded video to one isolated voice per visible face, built on lip-conditioned extraction;
- three signal-processing stages of our own design: coherence-protected refinement, coherent leak cancellation and a word-safe silence gate;
- an interactive player in which switching between speakers, and between strict and natural isolation, is a click-free crossfade on one audio clock;
- appearance re-identification, so that a person keeps their channel across camera cuts; and
- an evaluation method, including a ground-truth leak bench built from our own recording, that measures the leakage which word error rate alone hides.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The scope was fixed in the proposal and refined as measurements came in. Table 3 lists what the system must and need not handle, and Table 4 the success criteria used throughout the report.

In scope	Out of scope
Videos with two or more visible speakers (tested with up to three)	People who never appear on screen
One microphone: phone, webcam, television or web video	Microphone arrays and spatial cues
Any language (tested in English and Bengali)	Live, real-time streams (processing is offline; playback is interactive)
Process once, then play interactively with instant switching	Faces smaller than about 150 px, or turned fully away from the camera
Local processing on a laptop with an 8 GB GPU	Music and other non-speech sources

Table 3: Scope of the project.

Objective	Requirement	How it is measured
Isolation	The unselected voice should be inaudible, and there should be exact silence when the chosen person pauses.	Signal-to-interference ratio (SIR) on a ground-truth bench; similarity between channels; share of pause samples that are exactly 0.0.
Fidelity	The selected voice is unaltered: no clipped words and no words of the other speaker.	Word error rate (WER) and intruder words against the read scripts; share of word time muted by the gate.
Interaction	Switching within 25 ms without clicks; audio and video in sync.	Web Audio crossfade design; drift monitor in the player.
Generality	Works on videos other than the showcase clip.	Five test videos: two phone recordings, a split-screen news debate, a talk show and an edited talk show with camera cuts.
Practicality	Runs on an 8 GB laptop GPU within a few times real time.	Measured processing time.

Table 4: Success criteria.

3.1.2 Literature Review

Speech separation has a long history. The approaches relevant to this project fall into five groups, summarised in Table 5.

Classical multi-channel and statistical methods. Beamforming and independent component analysis (ICA) [2] separate sources by exploiting spatial diversity or statistical independence. They need at least as many microphones as sources; with the single channel of a phone or television recording the problem is underdetermined, so these methods do not apply.

Deep blind separation. Deep clustering [3] and permutation-invariant training (PIT) [4] made single-channel separation of unknown speakers possible by solving the label-permutation problem during training. Conv-TasNet [5] moved separation into the time domain with a learned encoder and decoder, and the dual-path transformer SepFormer [6] and MossFormer2 [7] raised benchmark performance further. These models output a fixed number of streams in an arbitrary order: they cannot say which stream belongs to which face, and the order can change between processing chunks. Our first version used a SepFormer trained on WHAMR! [8], whose training targets are anechoic while its inputs are reverberant; the model therefore also removes the room, which listeners hear as a change in the character of the voice.

Target-speaker extraction with an audio reference. SpeakerBeam [9] and VoiceFilter [10] condition the network on an embedding of a clean enrolment utterance of the target and output only that person. The permutation problem disappears, but a clean reference recording of every person is rarely available for an arbitrary video.

Audio-visual separation and extraction. Ephrat et al. [11] showed that face embeddings let a network pick out a visible speaker, and Afouras et al. [12] and Gao and Grauman [13] used lip motion and cross-modal consistency for the same purpose. The target-speaker extraction model of ClearerVoice-Studio, AV-MossFormer2 [14], combines a lip-reading visual front-end with the MossFormer2 separator and outputs a single stream for the face it is given; Dolphin [15] reduces the cost with discrete lip semantics. Because the lips choose the voice, these models need no matching stage and extend to any number of visible speakers. Their weak point is exactly the case our showcase recording targets: two similar voices that overlap continuously.

Supporting technology. Face detection comes from MediaPipe [16]; appearance re-identification uses SFace embeddings [17] with YuNet landmarks [18]; speaker embeddings use ECAPA-TDNN [19] from SpeechBrain [20]; and evaluation transcripts come from Whisper [21]. The signal-

processing building blocks — sampling-rate conversion, the short-time Fourier transform (STFT) and its overlap-add inverse [22], [23], Wiener filtering [24] and the discrete-time analysis taught in this course [25], [26] — are standard; our contribution is in how they are combined and tuned against measurements.

Gap addressed by this project. The published audio-visual systems report scores on synthetic mixtures and stop at an output file. None of the systems we found offers per-face interactive playback, follows people across camera cuts, removes the residual leak between face channels, or guarantees silence in pauses. This project addresses those gaps and evaluates them on real recordings, including our own.

Approach	Representative work	Idea	Limitation for this problem
Beamforming, ICA	[2]	Spatial diversity or statistical independence	Needs several microphones
Deep blind separation	[3], [4], [5], [6], [7]	Learn masks or waveforms for a fixed number of speakers	No face–voice identity; output order can swap; fails on similar voices
Extraction with enrolment	[9], [10]	Condition on a clean sample of the target voice	Clean enrolment rarely available
Audio-visual extraction	[11], [12], [13], [14], [15]	Condition on the target’s face or lips	Leaks when similar voices overlap continuously; no interactive use
This project	this report	AV extraction + refinement + leak cancellation + exact-silence gate + web player	Measured on our own recordings; limits in Section 5.4

Table 5: Summary of the literature.

3.1.3 Formulation of Problem

Let the recorded mixture be $x(t)$ as in Eq. (1), with K unknown sources $s_k(t)$ and noise $n(t)$, sampled at $f_s = 16$ kHz. Let $V_1, \dots, V_N$ be the mouth-region videos of the N faces visible in the recording ($N \leq K$). For every face i we seek an estimate

$$\hat{s}_i(t) = f(x(t), V_i), \quad i = 1, \dots, N \quad (2)$$

of that person’s voice, where f is the processing chain. The estimate is judged on three objectives:

1. **Isolation:** maximise the signal-to-interference ratio of every channel, Eq. (3) [27], and make $\hat{s}_i(t)$ exactly zero wherever person i is silent;
2. **Fidelity:** keep the selected voice intact — minimise the word error rate of a transcription of $\hat{s}_i$ against what person i said, and never mute a word;
3. **Interaction:** switch the audible channel within 25 ms, without a click and without losing audio–video synchronisation.

The signal-to-interference ratio of channel i is

$$\text{SIR}_i = 10 \log_{10} \frac{\|s_i^{\text{target}}\|^2}{\|e_i^{\text{interf}}\|^2} \text{ dB} \quad (3)$$

where s_i^{target} is the part of $\hat{s}_i$ that comes from person i and e_i^{interf} the part that comes from other people. This decomposition is available only when the sources are known, which is why we built the ground-truth bench of Section 3.3. The design is constrained to one microphone, a 16 kHz model, a laptop GPU with 8 GB of memory (RTX 5060 Laptop) and Python 3.12, and it must run locally so that no

recording leaves the user’s computer. Isolation and fidelity conflict: every stage that removes more of the other voice also risks removing some of the target, so each design decision below is a measured trade-off between the two.

3.1.4 Analysis

Three measurements, made before the final design was fixed, determined its shape.

(a) Matching faces to separated voices does not work. Our V2 system separated the audio blindly and then assigned each stream to a face by correlating a lip-opening signal with the stream’s loudness, followed by Hungarian assignment [28]. On a real talk-show clip the matching statistic was 0.0679; the same statistic computed against pure white noise was 0.0683, and a circular-shift permutation test gave $p = 0.71$ – 0.96 . The lip signal, one number per frame, tracked total speech activity rather than who was speaking: its correlation with the audio was about $+0.18$ for both faces. We therefore stopped matching after separation and conditioned the separation itself on the lips.

(b) Audio-only separation cannot split two similar voices. On our showcase recording each of SepFormer’s two outputs contained words from both texts, giving word error rates of 80–128 %. This was not a chunking error: the same interleaving occurred when the whole clip was processed in one call. Only the lips distinguish these two speakers.

(c) The extraction model must run in its training regime. The pretrained AV-MossFormer2 decodes 3 s windows in its reference implementation. Our first integration used 12–14 s windows to give the network more context, and on continuous overlap the two faces’ outputs became near-copies of the mixture. Figure 2 shows the similarity between the two channels (ECAPA cosine at the same instant) as the window shrinks: on the 20 s phone clip it falls from 0.86 at 14 s to 0.60 at 2 s, and on the 37 s clip from 0.71 to 0.52, while on the turn-taking talk show it stays near 0.12 at every length. Short windows are therefore better, and their extra cost is recovered by batching.

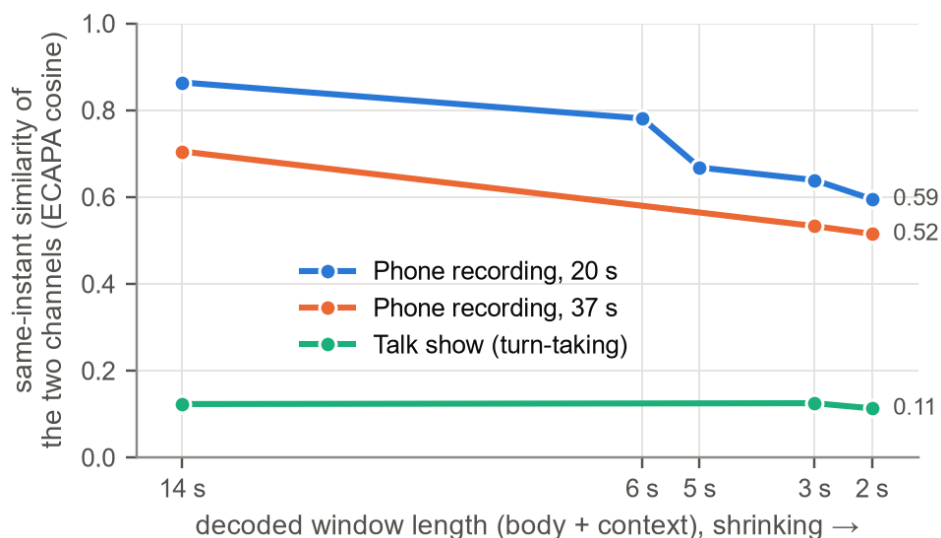


Figure 2: Similarity between the two isolated channels at the same instant versus decoding window length (lower is better; 0 = two different people, 1 = the same audio). Windows close to the model’s 3 s training windows separate the overlapped phone recordings much better and make no difference on the turn-taking talk show.

Two further findings shaped the evaluation rather than the design. First, word error rate hides leakage: Whisper transcribes the louder voice, so a channel can read cleanly while the other speaker is still audible. We therefore built a ground-truth leak bench (Section 3.3) to measure leakage directly.

Second, in the V2 prototype the faint residual of the other voice had been made loud by per-source peak normalisation: SpeechBrain’s `separate_file()` rescales every output to full scale, so a residual 25 dB below the speech was played as loud as speech. An amplitude bug, not a lack of filtering, caused the prototype’s “ghost whisper” — a reminder to verify the gain staging of a system before designing filters for it.

3.2 Design Method (PO(a))

This section describes each stage of the final system (Figure 3) and the mathematics behind it. Stages 1–4 and 8 integrate pretrained models and standard tools; stages 5–7 are the signal-processing stages we designed. Table 8 at the end of the section collects every parameter.

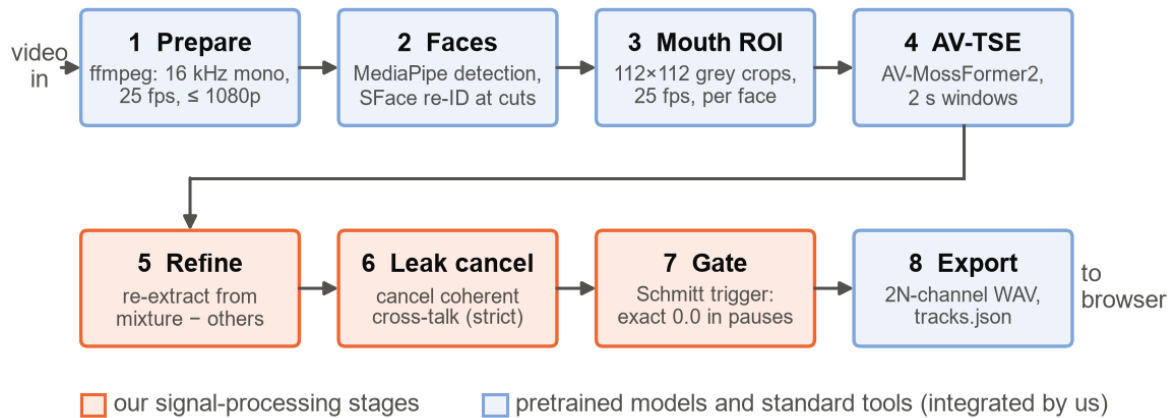


Figure 3: Processing pipeline. A video goes in; one isolated voice per face, in a strict and a natural version, goes to the browser player. The orange stages are our signal-processing design.

3.2.1 Pre-processing and sampling-rate conversion

ffmpeg decodes the upload into a mono 16 kHz audio track and a working video at 25 frames/s, capped at 1080p. Phone recordings are sampled at 48 kHz, so the audio is decimated by $M = 3$; before discarding samples the resampler low-pass filters the signal below the new Nyquist frequency of 8 kHz to prevent aliasing [25]. The video is converted to 25 frames/s because the extraction model was trained on 25 frames/s lip video with 16 kHz audio, so that each video frame corresponds to exactly

$$N_v = \frac{f_s}{f_v} = \frac{16000}{25} = 640 \text{ samples.} \quad (4)$$

All windowing downstream is counted in whole video frames, so the audio and the lips of every processing window start at the same instant. Converting the frame rate by repeating or dropping frames introduces at most half a frame (20 ms) of timing error.

3.2.2 Face tracking, re-identification and mouth regions

MediaPipe [16] detects up to four faces in every fifth frame, and an intersection-over-union tracker (threshold 0.3) links the detections into tracks. A tracker that assumes continuous motion fails at camera cuts, so each track also carries a 128-dimensional SFace appearance embedding [17], aligned with the five landmarks of YuNet [18]. A face that reappears after more than 12 frames (0.48 s) is re-joined to a track only if the cosine similarity of the embeddings exceeds OpenCV’s same-identity

threshold of 0.363, and a position-based match is refused if the cosine falls below 0.20. On a 196 s edited talk show this raised the share of the clip in which each person was tracked from 0.2–1.6 % to 44 %, 18 % and 8 %, with identities kept across cuts.

For every tracked face the mouth region is cut out exactly as the model’s visual front-end expects. With the box half-size $b = \max(w, h)/2$, a square of side $2.8b$ centred $0.4b$ below the centre of the box is resized to 224×224 pixels, converted to grey scale and centre-cropped to 112×112 — in effect a square of side $1.4b$ around the mouth and chin. Pixel values v are normalised with the statistics of the lip-reading data the front-end was trained on,

$$\hat{v} = \frac{v/255 - 0.4161}{0.1688} \quad (5)$$

and the 112×112 size is fixed by the network (four stride-2 stages followed by 4×4 average pooling). When a face is off screen for longer than 0.48 s its mouth image is frozen, its channel is muted for that time, and the corresponding windows are not processed at all.

3.2.3 Lip-conditioned extraction

The AV-MossFormer2 target-speaker extraction model [14] takes the mixture and one face’s mouth video and returns one waveform: that person’s voice. Its visual front-end (a 3-D convolution followed by a ResNet-18 lip-reading network) encodes each video frame as a 512-dimensional vector that describes the shape of the mouth, including the speech sound being formed, and a MossFormer2 masking network [7] uses that sequence to extract the matching voice. The network is called once per face, so channel i belongs to face i by construction: no permutation can occur between chunks, and there is no limit of two speakers. We vendored the model’s 60 KB of PyTorch [29] code (Apache-2.0) instead of installing its package, because the package pins NumPy below version 2 and would have forced a second Python environment. Two lines were changed: an import path, and a hard-coded `.cuda()` call that prevented running on a CPU.

Recordings are processed in overlapping windows. Each window has a kept body of $B = 25$ video frames (1 s) and a context margin of $C = 12$ frames (0.48 s) on each side that is decoded and then discarded, so every window the model sees is 49 frames (1.96 s) long, close to its 3 s training windows (Section 3.1.4). The model is least reliable at the edges of its input; discarding the margins keeps those edges out of the output. Adjacent bodies overlap by one video frame (40 ms) and are joined by a normalised overlap-add with raised-cosine fades,

$$\hat{s}(n) = \frac{\sum_k g_k(n) y_k(n)}{\sum_k g_k(n)}, \quad g(n) = \frac{1}{2} (1 - \cos(\pi n/L)) \quad (6)$$

where y_k is the kept body of window k and g_k its fade window, of length $L = 640$ samples at each join. The earlier design used a 2 s crossfade, which made 20 % of every output a blend of two independent inferences whose small phase differences comb-filtered the voice. Windows are run in batches whose size is derived from the free GPU memory M_{free} with a measured linear memory model,

$$M \approx 276 + 124(B + 2C) \text{ MiB}, \quad n_b = \left\lfloor \frac{0.5M_{\text{free}} - 276}{124(B + 2C)} \right\rfloor \quad (7)$$

with B and C in seconds; the batch is halved automatically after an out-of-memory error. TensorFlow-32 arithmetic makes inference 17 % faster while changing the output by a component 63 dB below the signal.

3.2.4 Refinement: subtracting the other speakers, not yourself

After the first pass every face has an estimate $\hat{S}_j(f, \tau)$ in the STFT domain,

$$S(f, \tau) = \sum_{n=0}^{N-1} s[n + \tau H] w[n] e^{-j2\pi f n/N}, \quad N = 512, \quad H = 128, \quad (8)$$

computed with a Hann window $w[n]$ of $N = 512$ samples (32 ms) and hop $H = 128$ (8 ms). At 75 % overlap the Hann window satisfies the constant-overlap-add condition, so the analysis–synthesis round trip reconstructs a signal to within 10^{-6} [22], [23].

A cleaner input for face i is the mixture minus the other faces' estimates. Plain subtraction failed, however: where face i speaks and face j is silent, the model conditioned on face j partly returns face i 's voice, and subtracting it cancels the target (on the 37 s clip, face A's word error rate rose from 2.3 % to 30.2 %). The part of $\hat{S}_j$ that is face i 's voice is recognisable because it is the same waveform as $\hat{S}_i$ and therefore phase-coherent with it. We estimate it by a local least-squares regression in every time–frequency cell,

$$a_{ij} = \arg \min_a \sum_N |\hat{S}_j - a \hat{S}_i|^2 = \frac{\sum_N \hat{S}_j \hat{S}_i^*}{\sum_N |\hat{S}_i|^2}, \quad |a_{ij}| \leq 1 \quad (9)$$

where the sums run over a neighbourhood $N(f, \tau)$ of 5 bins $\times$ 9 frames (156 Hz $\times$ 72 ms) around the cell and the asterisk denotes the complex conjugate. The neighbourhood holds 45 cells, so two independent voices look coherent only by chance, with a bias of about $1/\sqrt{45} \approx 0.15$. The magnitude of a_{ij} is clipped to 1 so that no more of $\hat{S}_j$ is attributed to face i than face i 's own estimate holds. The refined input for face i is

$$x'_i(t) = x(t) - \sum_{j \neq i} \text{ISTFT}\{\hat{S}_j - (1 - \lambda) a_{ij} \hat{S}_i\}, \quad \lambda = 0.5 \quad (10)$$

that is, everything in $\hat{S}_j$ that is not coherent with face i is subtracted in full, and the shared part only at weight λ , because the audio alone cannot tell whether it is face i 's voice leaking into face j 's estimate or the reverse. $\lambda = 0.5$ was the best of $\{0, 0.5, 1\}$ on both phone clips. The model is then run a second time on x'_i . Table 6 shows the effect.

Method	Phone 20 s: A / B	Phone 37 s: A / B
One pass, no refinement	10.9 / 36.2 %	2.3 / 7.6 %
Plain subtraction of the other estimate	17.4 / 19.0 %	30.2 / 19.6 %
Coherence-protected, $\lambda = 0.5$ (used)	2.2 / 24.1 %	3.5 / 5.4 %

Table 6: Word error rate of each face's channel with and without refinement (lower is better). Plain subtraction cancels the target; protecting the coherent part fixes it. Refinement costs face A of the 37 s clip 1.2 points but gains much more elsewhere; a second refinement pass gave no further improvement. Face B's script for the 20 s clip is partly reconstructed, so its error floor is not zero.

3.2.5 Coherent leak cancellation (strict isolation)

Even after refinement each channel contains a quieter copy of the other voice. What leaks into channel i from face j is a copy of face j 's waveform, so it is phase-coherent with channel j , while face i 's own speech is not. For every pair (i, j) we regress channel i onto channel j over the same neighbourhood and subtract the coherent component, weighted by which channel owns it:

$$\hat{S}_i \leftarrow \hat{S}_i - \sum_{j \neq i} w_{ij} c_{ij} \hat{S}_j, \quad c_{ij} = \frac{\sum_N \hat{S}_i \hat{S}_j^*}{\sum_N |\hat{S}_j|^2}, \quad |c_{ij}| \leq 1 \quad (11)$$

$$w_{ij} = \frac{1}{1 + (P_i/P_j)^\gamma}, \quad P_i = \sum_N |\hat{S}_i|^2, \quad \gamma = 1 \quad (12)$$

A component shared by both channels belongs to the channel that holds it louder: when $P_j \gg P_i$ the weight w_{ij} tends to 1 and the component is removed from channel i ; when $P_i \gg P_j$ it tends to 0 and channel i keeps it. The weight has the form of a Wiener gain [24] applied to the coherent part only. Independent speech is not coherent ($c_{ij} \approx 0$) and is left untouched, which is why this step costs far less clarity than a power mask. On the leak bench it raised the signal-to-interference ratio from 10.4/8.1 dB to 16.3/13.4 dB (Section 3.3), at a cost of 2.3 and 1.1 percentage points of word error rate on the 37 s clip. Because the listener may want to make this trade-off differently, both versions are exported and the player switches between “strict” and “natural” isolation during playback; strict is the default.

3.2.6 Word-safe silence gate

A separation network trained with a scale-invariant loss attenuates the other voice but never outputs exact silence, so silence has to be imposed after resynthesis, in the time domain. Zeros set in the STFT domain do not survive, because the overlap-add of the inverse STFT spreads every zeroed frame by $N - H = 384$ samples (24 ms) on each side. The gate computes the level of each 10 ms frame relative to the channel’s own 95th-percentile level L_{95} ,

$$L(m) = 10 \log_{10} \left(\frac{1}{L_f \sum_{n=0}^{L_f-1} x^2[mL_f + n]} \right) - L_{95}, \quad L_f = 160, \quad (13)$$

so the thresholds do not depend on the recording level, and drives a Schmitt trigger: a two-threshold state machine with hysteresis and minimum dwell times,

$$q(m) = \begin{cases} 1 & \text{if } L(m) > \theta_{on} \text{ for } \geq 60 \text{ ms} \\ 0 & \text{if } L(m) < \theta_{off} \text{ for } \geq 120 \text{ ms} \\ q(m-1) & \text{otherwise} \end{cases} \quad (14)$$

with $\theta_{on} = -30$ dB and $\theta_{off} = -40$ dB. When the gate opens, the opening is applied retroactively over the 60 ms that justified it, so word onsets are not cut, and the 10 dB hysteresis band stops the gate chattering on the dips between syllables. The binary state is widened by a 10 ms look-ahead and converted into a gain $g(n)$ with 10 ms raised-cosine ramps,

$$g_r(n) = \frac{1}{2} (1 - \cos(\pi n/R)), \quad R = 160, \quad y(n) = g(n) \hat{s}(n), \quad (15)$$

which reach exactly 0 and exactly 1: the output is bit-exact zero wherever the gate is closed, and no transition clicks. The thresholds were re-fitted for this path against Whisper word timings (Table 7). The V2 setting (−20/−23 dB, 10 ms) had been fitted to squeeze SepFormer’s residual; on continuous overlapped reading it closed inside words. The gate always listens to the natural channel and is applied to both the natural and the strict version, because listening to the processed audio cut 2–17 % of word time. Figure 4 shows the gate on the man’s channel of the news debate: it stays closed while he is silent (for example from 9.1 s to 12.6 s, while the woman speaks), and 32 % of the channel’s samples are exactly 0.0.

Gate setting	Word time muted	Gaps between words set to zero
V2: -20/-23 dB, 10 ms hold	12.2–17.7 %	37–72 %
Final: -30/-40 dB, 120 ms hold	0.05–1.6 %	0.1–29 %

Table 7: Gate settings measured against Whisper word timings on the two phone recordings (four channels). The gaps between words in these recordings are not silent — they hold the other reader a few dB lower — so no threshold can zero them without cutting words.

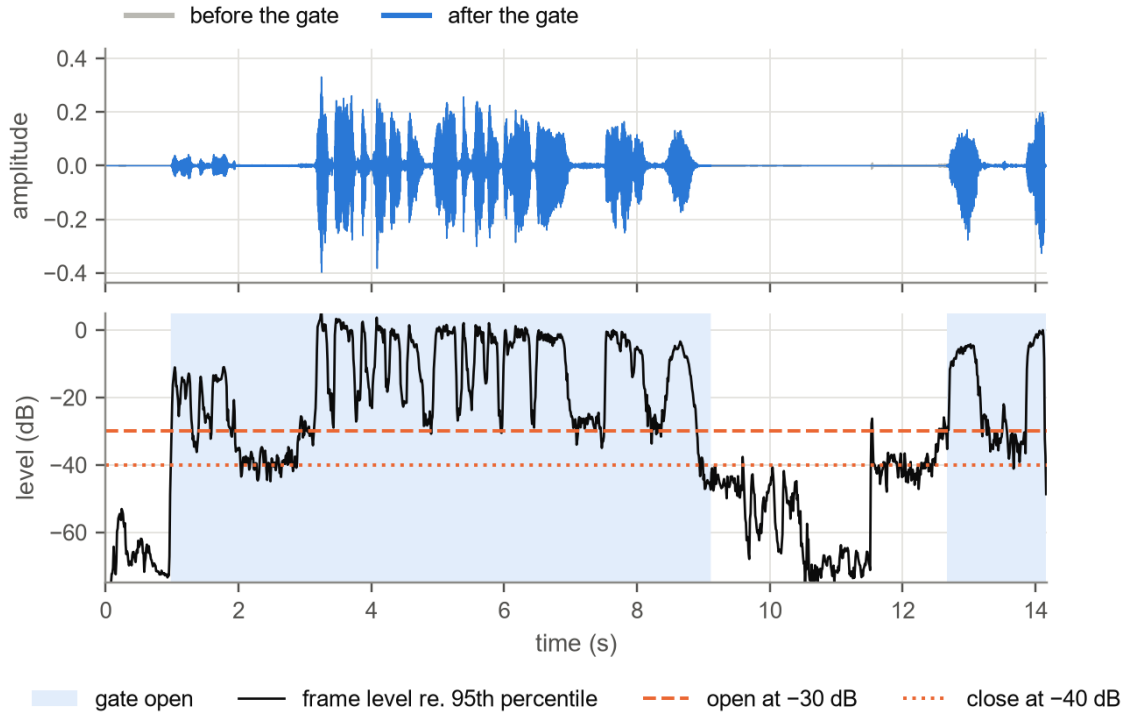


Figure 4: The silence gate on real output (news debate, channel A, natural version). Top: waveform before (grey) and after (blue) the gate. Bottom: frame level relative to the channel's 95th percentile, the two thresholds, and the regions where the gate is open (shaded).

3.2.7 Interactive player

Heavy inference cannot run at click time, so the backend precomputes every face's voice and the browser only chooses which one is heard. The design goal is a single audio clock (Figure 5), built with the Web Audio API [30]. All $2N$ channels (N strict and N natural) are delivered in one lossless 16 kHz WAV file; lossy coding would add noise precisely in the silent regions that the gate creates. The file is decoded once into an audio buffer and played by a single `AudioBufferSourceNode`; a `ChannelSplitterNode` fans it out to one `GainNode` per channel, and exactly one gain is 1 at any time. Because every channel plays from the same buffer and source, the channels cannot drift relative to each other. The `AudioContext` runs at 16 kHz, so the browser does not resample the audio.

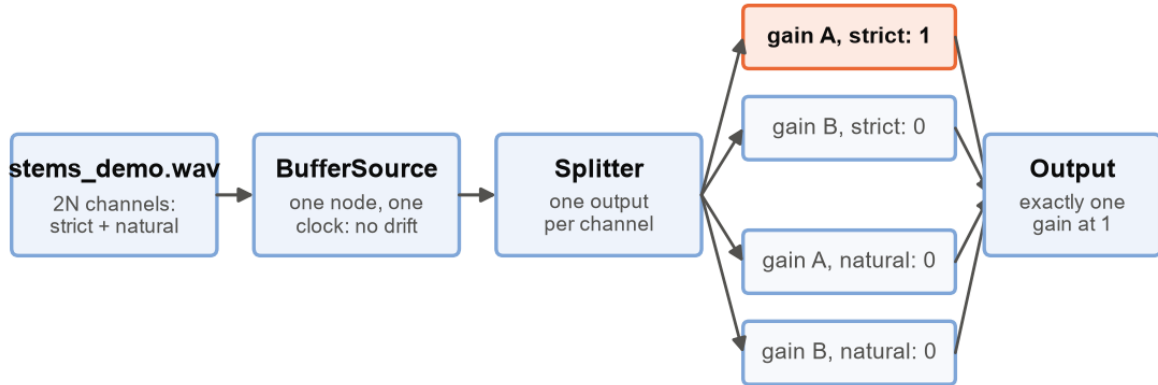
Clicking a face schedules a 25 ms equal-power crossfade on the audio thread (`setValueCurveAtTime`):

$$g_{in}(t) = \sin(\pi t/2T), \quad g_{out}(t) = \cos(\pi t/2T), \quad g_{in}^2(t) + g_{out}^2(t) = 1, \quad T = 25 \text{ ms} \quad (16)$$

Because the powers of the two gains always sum to one there is no dip in loudness, and because the gain curve is smooth there is no click. Switching between strict and natural isolation is the same crossfade to the other set of channels. The video element is muted and follows the audio clock: at every video frame (`requestVideoFrameCallback`) the drift $d = t_{\text{video}} - t_{\text{audio}}$ is measured. Within ± 20 ms nothing is done; up to 250 ms the playback rate of the video is corrected proportionally,

$$r = 1 - \text{clip}(0.5(d - 0.02 \text{sgnd}), -0.02, 0.02) \quad (17)$$

and beyond 250 ms the video is re-seeked. The audio rate is never changed, since that would shift the pitch. Level meters are tapped after the gains, so a muted speaker shows a flat zero.



Click a face: 25 ms equal-power crossfade ($\cos^2\theta + \sin^2\theta = 1$), so no dip and no click.

Isolation strict $\leftrightarrow$ natural: the same crossfade to the other set of channels.

The <video> element is muted and slaved to the audio clock ($\pm 2\%$ rate, re-seek beyond 250 ms).

Figure 5: The browser player. One buffer and one source give one clock; a splitter and one gain per channel select what is heard. Switching speaker or isolation mode is an equal-power crossfade of the gains.

3.2.8 Software architecture

The backend is a FastAPI application served by uvicorn on 127.0.0.1:8000. An upload creates a job with a server-generated identifier; the job runs on a single GPU worker thread, so two jobs can never exhaust the GPU memory together, and reports its stage and percentage to the browser through server-sent events. Each job writes the working video, stems_demo.wav (gated, 2N channels, played by the user interface), stems_raw.wav (ungated, used for objective scoring), tracks.json (face boxes for every frame, normalised to [0, 1], and the channel layout) and meta.json (parameters and measurements). Writing the gated and ungated versions from the same run guarantees that the scores and the demonstration come from the same processing. The earlier SepFormer path is kept as a fallback separator behind the same interface.

Stage	Parameter	Value	Reason
Pre-processing	Audio rate / video rate	16 kHz mono / 25 frames/s (640 samples per frame)	Fixed by the model's training data
Mouth region	Crop, size, normalisation	1.4b square, 0.4b below box centre; 112×112 grey; $(v/255 - 0.4161)/0.1688$	Visual front-end contract
Extraction	Kept body / context / join	25 / 12 / 1 frames (1 s / 0.48 s / 40 ms)	Short training windows (Figure 2)
Extraction	Batch size	From free GPU memory, at most 64, halved on OOM	Eq. (7)
STFT	Window, length N, hop H	Hann, 512 (32 ms), 128 (8 ms)	75 % overlap, perfect reconstruction
Refinement	Neighbourhood / λ / passes	5 bins $\times$ 9 frames / 0.5 / 1	Best of {0, 0.5, 1}; a second pass gave no gain
Leak cancellation	Ownership exponent γ / extra mask	1 / off	The mask cost clarity (Table 9)
Gate	θ_{on} / θ_{off} / reference	-30 dB / -40 dB / 95th percentile	Word-safe (Table 7)
Gate	Minimum open / minimum close / look-ahead / ramp	60 / 120 / 10 / 10 ms	Onsets kept, no chatter, no clicks

Stage	Parameter	Value	Reason
Player	Crossfade / dead band / rate authority / re-seek	25 ms / 20 ms / $\pm 2\%$ / 250 ms	Click-free; pitch unchanged

Table 8: Design parameters of the final system.

3.3 Simulation Model

The showcase recording has no clean reference: both people read throughout, so the true voices are unknown and leakage cannot be measured on the recording directly. We therefore built a ground-truth leak bench from the recording itself (Figure 6a). Face A’s separated channel at time t and face B’s separated channel shifted to $t + 18$ s are added to form a synthetic mixture whose two components are known exactly; the 18 s shift makes the words unrelated, and each component keeps its own lip video. The mixture is passed through the pipeline, each output channel is decomposed into its target and interference components, and the signal-to-interference ratio of Eq. (3) is computed. A second case mixes face A with a shifted copy of himself, the hardest case of two identical voices.

Figure 6b shows the measured ratios. A single pass of the extraction model leaves the other voice only 9.9 dB (face A) and 6.9 dB (face B) below the target — the “huge leaks” we heard when listening. Refinement adds 0.5–1.2 dB, and coherent leak cancellation another 5.3–5.9 dB, reaching 16.3/13.4 dB. Table 9 lists the other approaches we simulated on the bench.

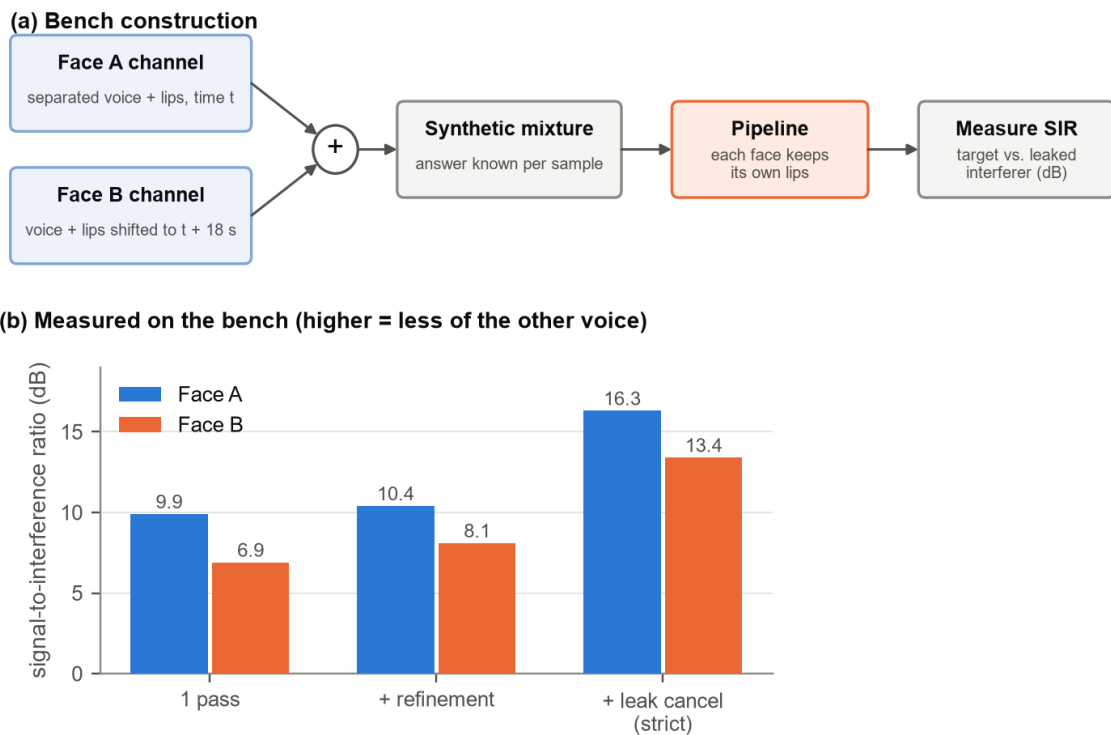


Figure 6: (a) Construction of the ground-truth leak bench from our own recording. (b) Signal-to-interference ratio after each stage.

Approach	Bench SIR, A / B	Decision
One pass of AV-MossFormer2	9.9 / 6.9 dB	Baseline
+ coherence-protected refinement	10.4 / 8.1 dB	Used
+ coherent leak cancellation	16.3 / 13.4 dB	Used as the “strict” mode
+ cross-face ratio mask ($p = 4$)	18.4 / 15.9 dB	Rejected: WER of face A on the 37 s clip rose from 3.5 % to 23.3 %, and on the talk show the channels became less distinct (0.13 $\rightarrow$ 0.20)
Dolphin (ICLR 2026) instead of AV-MossFormer2	6.5–8 / 0–3 dB	Rejected: its landmark-aligned crop assumes a frontal face, and face B reads looking down
A third extraction pass on masked input	no gain	Rejected
Averaging over mouth-crop augmentations	no gain	Rejected

Table 9: Approaches simulated on the leak bench.

Earlier, the V2 silence chain had been tuned on a synthetic bench of formant-synthesised speech with artificial leakage. It showed that a Wiener mask applied to the separated estimates with exponent $p = 2$ is the knee of the trade-off (leakage of -19.4 , -35.8 and -41.1 dB for $p = 1, 2$ and 3), that a conventional -20 dB mask floor collapses the suppression to -1.6 dB, and that masking the estimates is 12–17 dB better than masking the mixture. These results still govern the SepFormer fallback path. They also taught us that absolute thresholds fitted on synthetic signals do not transfer to real recordings, which is why every threshold of the final system was re-fitted on real clips. For the same reason the mixture-consistency projection [31] was not used: the WHAMR! model is trained to remove noise and reverberation, so its outputs do not sum to the mixture, and forcing them to would reinject exactly the residual we wanted to remove.

3.4 Source Code of the Core Signal-Processing Routines

The complete source code — about 18 000 lines of Python in the application and 45 diagnostic and verification scripts, plus the JavaScript player — is in the project repository (Section 7.3). Table 10 lists the routines that implement our three signal-processing stages, condensed from `app/separation.py` and `app/dsp.py` (documentation removed, logic unchanged).

<pre> # Refinement input per face, Eqs. (9)-(10) def refinement_inputs(mix, stems, sr, lam=0.5, size=(5, 9)): sft = stft_engine(nfft=512, hop=128, fs=sr) S = [sft.stft(s) for s in stems] out = np.empty_like(stems) for i, own in enumerate(S): pw = local(abs(own)**2, size) + 1e-12 remove = np.zeros_like(own) for j, Sj in enumerate(S): if j == i: continue a = local(Sj * own.conj(), size) / pw m = np.abs(a) # clip a to 1 a = np.where(m > 1, a / m, a) remove += Sj - (1 - lam) * a * own y = sft.istft(remove, k1=len(mix)) out[i] = mix - y[:len(mix)] return out # Leak cancellation (strict), Eqs. (11)-(12) def strict_isolation(stems, sr, size=(5, 9), g=1.0): sft = stft_engine(nfft=512, hop=128, fs=sr) S = [sft.stft(s) for s in stems] P = [local(abs(s)**2, size) + 1e-20 for s in S] out = np.empty_like(stems) for i in range(len(S)): y = S[i].copy() for j in range(len(S)): if j == i: continue c = local(S[i] * S[j].conj(), size) c = c / P[j] m = np.abs(c) c = np.where(m > 1, c / m, c) w = 1 / (1 + (P[i] / P[j]) ** g) y -= w * c * S[j] out[i] = sft.istft(y, k1=stems.shape[1]) return out def local(z, size): # box average, 5 x 9 from scipy.ndimage import uniform_filter as uf if np.iscomplexobj(z): re = uf(z.real, size) return re + 1j * uf(z.imag, size) return uf(z, size) def advance(st, k): # 10 ms look-ahead out = st.copy() for d in range(1, k + 1): out[:-d] = st[d:] return out </pre>	<pre> # Schmitt-trigger gate, Eqs. (13)-(15) def gate_mask(x, sr, open_db=-30, close_db=-40, min_on_ms=60, min_off_ms=120, ramp_ms=10, frame_ms=10, look_ms=10): f = round(sr * frame_ms / 1000) # 160 nf = -(-len(x) // f) e = np.pad(x, (0, nf * f - len(x))) e = (e.reshape(nf, f) ** 2).mean(axis=1) ldb = 10 * np.log10(e + 1e-12) rel = ldb - np.percentile(ldb, 95) st = schmitt(rel, open_db, close_db, round(min_on_ms / frame_ms), round(min_off_ms / frame_ms)) st = advance(st, round(look_ms / frame_ms)) env = np.repeat(st.astype(np.float32), f) return ramp(env[:len(x)], sr, ramp_ms) def schmitt(rel, on_th, off_th, min_on, min_off): st = np.zeros(len(rel), dtype=bool) is_open, dwell = False, 0 for i, v in enumerate(rel): if is_open: dwell = dwell + 1 if v < off_th else 0 if dwell >= min_off: is_open, dwell = False, 0 else: dwell = dwell + 1 if v > on_th else 0 if dwell >= min_on: is_open, dwell = True, 0 lo = max(0, i - min_on + 1) st[lo:i + 1] = True st[i] = is_open return st def ramp(env, sr, ramp_ms): # raised cosine L = round(sr * ramp_ms / 1000) # 160 k = np.arange(L) up = 0.5 * (1 - np.cos(np.pi * k / L)) out = env.copy() for e in np.flatnonzero(np.diff(env)) + 1: if env[e] > env[e - 1]: # rising s = max(0, e - L) seg = up[L - (e - s):] out[s:e] = np.maximum(out[s:e], seg) else: # falling t = min(len(env), e + L) seg = up[:t - e] out[e:t] = np.maximum(out[e:t], seg) return np.clip(out, 0.0, 1.0) </pre>
---	--

Table 10: Source code of the core signal-processing routines (condensed from *app/separation.py* and *app/dsp.py*).

4 Implementation

4.1 Description

The system is a local web application. Table 11 lists the hardware and software used and Table 12 the code modules.

Component	Used in this project
Computer	Laptop with an NVIDIA GeForce RTX 5060 Laptop GPU, 8 GB (Blackwell, sm_120), Windows 11; earlier development on a GTX 1050 Ti (4 GB)
Runtime	Python 3.12; PyTorch 2.11 and torchaudio 2.11 (CUDA 12.8 build)
Models	AV-MossFormer2 TSE 16 kHz (vendored); SFace and YuNet (OpenCV); ECAPA-TDNN and SepFormer (SpeechBrain; diagnostics and fallback)
Vision	MediaPipe 0.10.21, OpenCV
Audio input/output and DSP	ffmpeg, soundfile, NumPy, SciPy (ShortTimeFFT)
Server	FastAPI, uvicorn, server-sent events
Client	HTML, CSS and JavaScript with the Web Audio API (no framework)
Evaluation	Whisper large-v3-turbo (word error rate), ECAPA-TDNN (channel similarity), leak bench and 45 diagnostic scripts

Table 11: Hardware and software.

Module	Role
app/config.py	Every tunable number, each with the measurement that justified it
app/media.py	ffmpeg decoding, resampling and multichannel WAV output
app/vision.py, app/reid.py	Face detection, tracking and SFace/YuNet re-identification
app/roi.py	112×112 mouth-region extraction matching the model’s training contract
app/avtse/	Vendored AV-MossFormer2 model code and weight loader
app/separation.py	Windowed, batched extraction per face; refinement; SepFormer fallback
app/dsp.py	Leak cancellation, Schmitt gate, Wiener mask, leakage and silence measurements
app/pipeline.py	Orchestration of the eight stages; writes both outputs
app/jobs.py, app/main.py	Job store, single GPU worker, FastAPI routes and progress events
app/static/	Player: upload page, video with face overlay, Web Audio engine

Table 12: Code modules.

Processing time was measured on five test videos with the final configuration (Table 13). The real-time factor lies between 1.1 and 2.5; the edited 196 s talk show is the fastest per second of video because windows in which a face is off screen are skipped.

Video	Length	Working resolution	Faces	Processing	Real-time factor
Split-screen news debate	14.2 s	1280×720	2	34.7 s	2.45
Bengali talk show (turn-taking)	32.3 s	1920×1080	2	54.7 s	1.69
Our phone recording, 20 s	20.5 s	1920×1080	2	42.7 s	2.08
Our phone recording, 37 s	37.5 s	1920×1080	2	74.4 s	1.98
Edited Bengali talk show with camera cuts	196.3 s	1920×1080	3	209.7 s	1.07

Table 13: Processing time on the RTX 5060 Laptop GPU. The phone recordings are 4K HEVC at 60 frames/s and are processed at 1080p and 25 frames/s.

Figure 7 shows the player on the best of the five test videos, a split-screen news debate in which a man and a woman talk over each other. Clicking a face, or its button, selects that speaker: the selected face is outlined in green, and the level meter under the button moves only for the selected speaker. The “Isolation: strict” button toggles to natural isolation and back, and “New video” returns to the upload page.

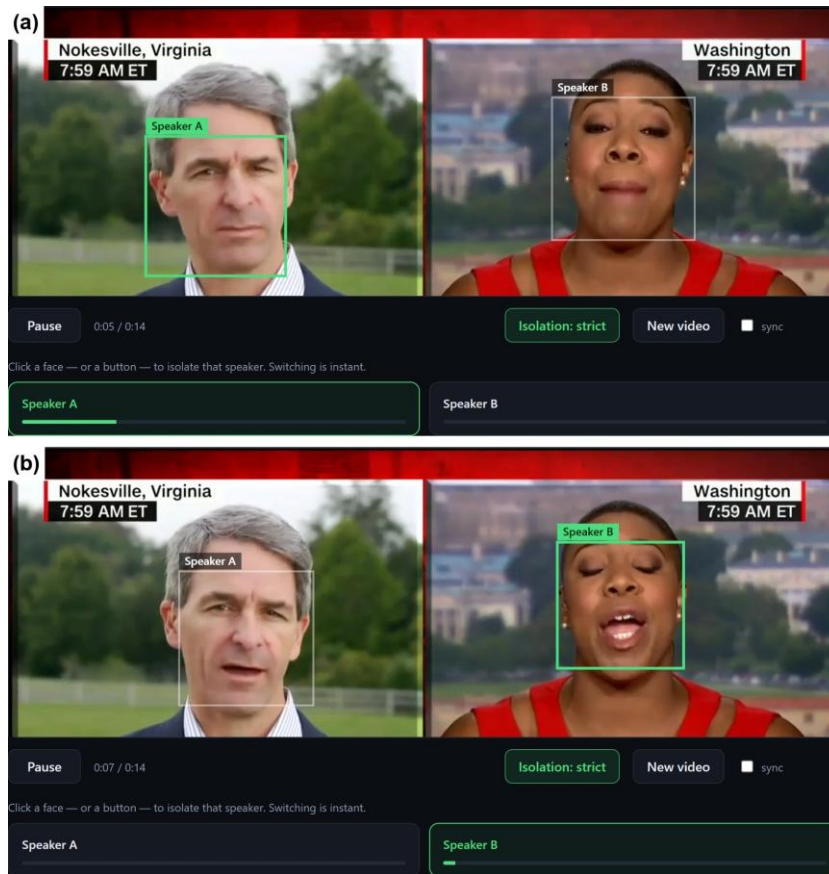


Figure 7: The web player on the news debate: (a) Speaker A selected, (b) Speaker B selected.

Two checks confirm that each channel belongs to the right face. Figure 8 shows the Pearson correlation

$$\rho = \frac{\sum_m (v_m - \bar{v})(e_m - \bar{e})}{\sqrt{\sum_m (v_m - \bar{v})^2 \sum_m (e_m - \bar{e})^2}} \quad (18)$$

between each face's lip motion v_m (the frame-to-frame change of the mouth image) and the loudness e_m of each channel, in video frame m . The diagonal dominates: face A's lips correlate with channel A at 0.78 and with channel B at only 0.17, and face B's lips correlate with channel B at 0.32 and with channel A at 0.14. Both faces also correlate less with the original mixture (0.39 and 0.26) than with their own channel, and the similarity between the voices of the two channels is only 0.08. Figure 9 shows the same relation over time: channel A falls silent while the man's lips are still, between 9 s and 12.5 s, and rises again with them.

Face A lips	+0.78	+0.17	+0.39
Face B lips	+0.14	+0.32	+0.26
	Isolated channel A	Isolated channel B	Original mixture

Figure 8: Face $\times$ voice correlation on the news debate (Pearson ρ between lip motion and channel loudness, 25 frames/s). Each face moves with its own isolated channel.

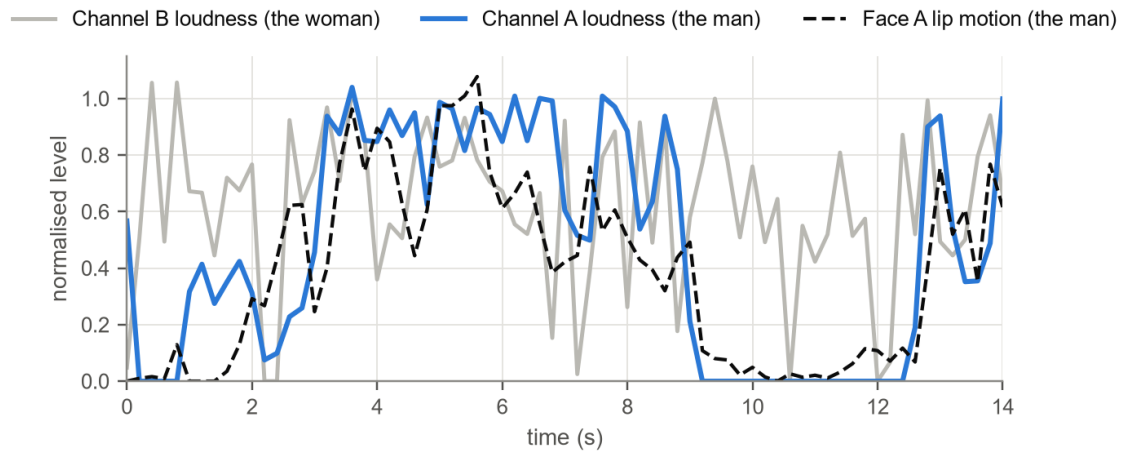


Figure 9: Face A's lip motion follows the loudness of channel A (the man), not that of channel B (the woman). All curves are normalised.

Figure 10 compares spectrograms (0–4 kHz) of the mixture and the two isolated channels over the same 14 s. Channel A is black — exactly silent — between the man's interjections, while channel B keeps the woman's continuous speech with its harmonic structure intact.

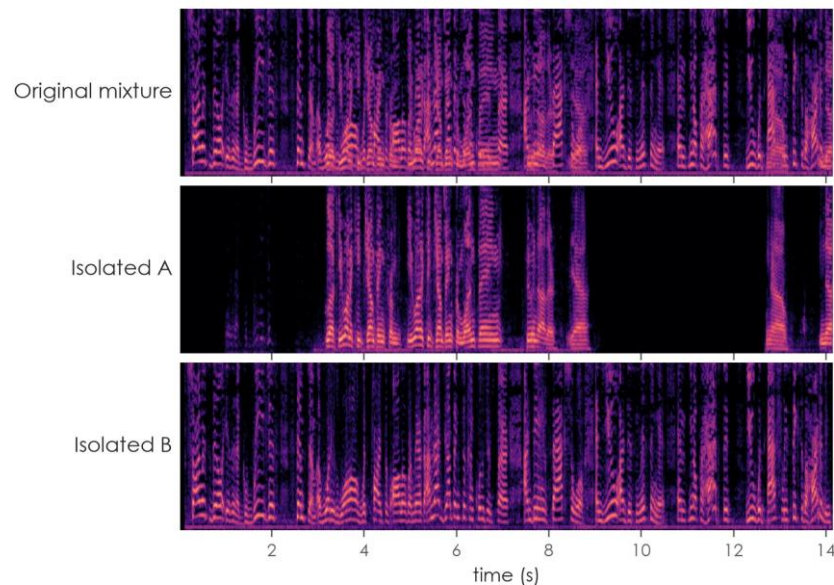


Figure 10: Spectrograms of the original mixture and the two isolated channels (news debate, 0–4 kHz). The gaps in channel A are exact digital silence.

The system is not tuned to one clip. Figure 11 shows it on our own phone recording and on an edited Bengali talk show with three people and frequent camera cuts, in which every person keeps the same label in every shot.



Figure 11: (a) Our phone recording with Speaker A isolated. (b) An edited Bengali talk show; three people are tracked. (c) Re-identification across camera cuts: the same person keeps the same label at 1, 11, 21, 32 and 42 s.

5 Design Analysis and Evaluation

5.1 Novelty

The individual building blocks exist in the literature; the novelty lies in the following combination and in the measurements that justified it.

1. **Lip-conditioned extraction in place of a measured failure.** We replaced a face–voice matcher that we had shown to perform at chance with per-face extraction, which removes the permutation, matching and two-speaker problems at once.
2. **Coherence-protected refinement.** Subtracting other speakers' estimates from the mixture normally cancels part of the target; regressing out the phase-coherent component (Eq. (9)) keeps the target and makes refinement work (Table 6).
3. **Coherent leak cancellation with a live strict/natural switch.** Leakage is removed only where it is a copy of another channel, and both versions are delivered in one buffer, so the listener chooses the balance between isolation and clarity during playback.
4. **Robustness to edited video.** Appearance re-identification keeps identities across camera cuts, and faces that are off screen are muted and not processed, which cut the computation on the 196 s talk show by 52 %.
5. **A ground-truth leak bench built from our own recording,** which revealed leakage that the transcripts hid and allowed every refinement to be measured rather than judged by ear alone.

Table 14 shows how the challenges we reported at the progress update were resolved.

Challenge at the progress update (7 Sep 2026)	Resolution in the final design
Only two speakers	N faces, one extraction per face (tested with three)
Voice shifting mid-clip	Face-conditioned output: channel i is face i by construction, so there is no permutation to drift; the new model keeps the room instead of drying the voice
Leftover background voice	Refinement, coherent leak cancellation and an exact-zero gate
Lip-tracking accuracy	Frame-aligned 112×112 mouth regions that match the model's training contract
Face leaving the frame	SFace re-identification across cuts; faces muted while off screen

Table 14: Progress-update challenges and their resolution.

5.2 Design Considerations (PO(c))

Beyond the technical objectives, the design took the following considerations into account.

5.2.1 Considerations to public health and safety

- **Privacy by design.** Everything runs on the user's computer; no audio or video is sent to any external service. The server listens only on 127.0.0.1, because it is an unauthenticated upload endpoint that must not be exposed on a shared network.
- **Safe input handling.** Uploads are limited to 512 MB and 5 minutes, only video file types are accepted, and the server generates the job identifier, so a user-supplied file name is never used to build a file path.
- **Hearing safety.** A single global gain is applied at export, which preserves the natural relative levels of the voices and prevents clipping. Every switch is a 25 ms equal-power crossfade and every gate transition a 10 ms ramp, so there are no sudden loud transients or clicks.
- **Listening comfort.** Pauses are exact digital silence rather than a faint, tiring residue of the other voice.

5.2.2 Considerations to environment

- Processing uses about 3 Wh of GPU energy per minute of video (Section 5.6), once per video; playback afterwards costs almost nothing.
- Windows in which a face is off screen are skipped: on the 196 s edited talk show the processing time fell from 436 s to 210 s (−52 %).
- TensorFlow-32 inference cut the processing time by 17 % with no audible change (a difference 63 dB below the signal).
- No model was trained; open pretrained models were reused, avoiding the large energy cost of training.
- The system runs on hardware the team already owned, so no new equipment was bought.

5.2.3 Considerations to cultural and societal needs

- **Language independence.** The visual cue is the shape of the mouth, not a vocabulary; the system was tested in English and Bengali, which matters in Bangladesh, where many recordings mix

both languages.

- **Accessibility.** The main beneficiaries are listeners who struggle to follow overlapping speech, including hearing-impaired and non-native listeners. Every face also has a keyboard-accessible button, and the keys 1–9 select speakers.
- **Inclusive testing.** The test material covers male and female voices, two similar male voices, and phone and broadcast video.
- **Respect for bystanders.** Channels are created only for visible faces; the system does not try to recover the voice of a person who never appears on screen.

5.3 Investigations (PO(d))

We treated the design as a series of experiments: every change was measured on the same clips before it was kept.

5.3.1 Design of Experiment

The investigation addressed five questions (Table 15).

Research question	Independent variable	Measured by
RQ1: Can audio-only separation isolate two similar overlapping voices?	Separator: SepFormer or AV-TSE	Word error rate and intruder words
RQ2: Which decoding window suits the pretrained model?	Window of 14, 6, 5, 3 or 2 s	Similarity between the channels at the same instant
RQ3: Do refinement and leak cancellation reduce leakage without harming clarity?	Refinement mode; strict or natural	Bench SIR; word error rate
RQ4: Can the gate produce silence without cutting words?	Thresholds and hold time	Word time muted; pause samples exactly zero
RQ5: Does the design generalise?	Five different videos	Channel similarity; correlation map; processing time

Table 15: Research questions.

The same clips, the same Whisper model (large-v3-turbo) and the same scoring scripts were used for every configuration, and the regression clips were re-run after every change. Objective scores were computed on the ungated output (`stems_raw.wav`), because a hard gate zeroes frames that a reference still contains and would distort the scores. The word error rate is

$$\text{WER} = \frac{S + D + I}{N_{\text{ref}}} \times 100 \% \quad (19)$$

where S, D and I are the substituted, deleted and inserted words and N_{ref} is the number of words in the script; it can exceed 100 % when a channel contains many words that are not in the script. Intruder words are words of the other speaker's script that appear in a channel. The similarity of two channels is the cosine of their ECAPA-TDNN speaker embeddings [19] computed over the same time windows,

$$\text{sim}(e_1, e_2) = \frac{e_1 \cdot e_2}{\|e_1\| \|e_2\|} \quad (20)$$

averaged over the clip; a value near 0 means that the two channels hold different people and a value near 1 that they hold the same audio.

5.3.2 Data Collection

Video	Source	Length	Language	Speakers and difficulty
Phone recording, 20 s	Our own (iPhone, 4K HEVC, 60 frames/s, 48 kHz mono)	20.5 s	English	Two similar male voices reading different texts continuously; known scripts
Phone recording, 37 s	Our own, same session	37.5 s	English	As above; longer
Split-screen news debate	Broadcast (CNN)	14.2 s	English	A man and a woman interrupting each other
Talk show	Broadcast	32.3 s	Bengali	A man and a woman on screen and a third man who is never shown; turn-taking
Edited talk show	Broadcast (Star News)	196.3 s	Bengali	Three people and many camera cuts

Table 16: Test videos.

The ground truth for the phone recordings is the two texts that were read, summaries of *Game of Thrones* (face A) and *House of the Dragon* (face B). Part of face B’s script for the 20 s clip had to be reconstructed, so that channel’s error floor is not zero. The broadcast clips have no reference; for them we used channel similarity, the lip–voice correlation map and listening. Recordings of people other than the team come from public broadcasts and were used only for internal testing.

5.3.3 Results and Analysis

RQ1 — separator. Table 17 and Figure 12 compare the separators. Audio-only separation produced word error rates above 100 % because each channel contained words from both texts (8 and 14 intruder words on the 37 s clip). Lip-conditioned extraction reduced the error by more than an order of magnitude, and our final natural output reads at 3.5 % and 5.4 % with no intruder words. The unrefined 12 s configuration transcribed face A slightly better, but its channels were near-copies of the mixture and contained intruder words: word error rate rewards a channel that holds both voices, as long as the target is the louder one.

System	Phone 20 s: A / B	Phone 37 s: A / B	Intruder words (20 s; 37 s)
Audio-only (SepFormer)	80 / 100 %	110.5 / 128.3 %	both texts interleaved; 8 + 14
AV-TSE, 12 s windows, no refinement	0 / 24 %	2.3 / 15.2 %	4; 6
Ours, natural	4.3 / 24.1 %	3.5 / 5.4 %	0; 0
Ours, strict	15.2 / 27.6 %	5.8 / 6.5 %	not measured

Table 17: Word error rate of each isolated channel against the read scripts.

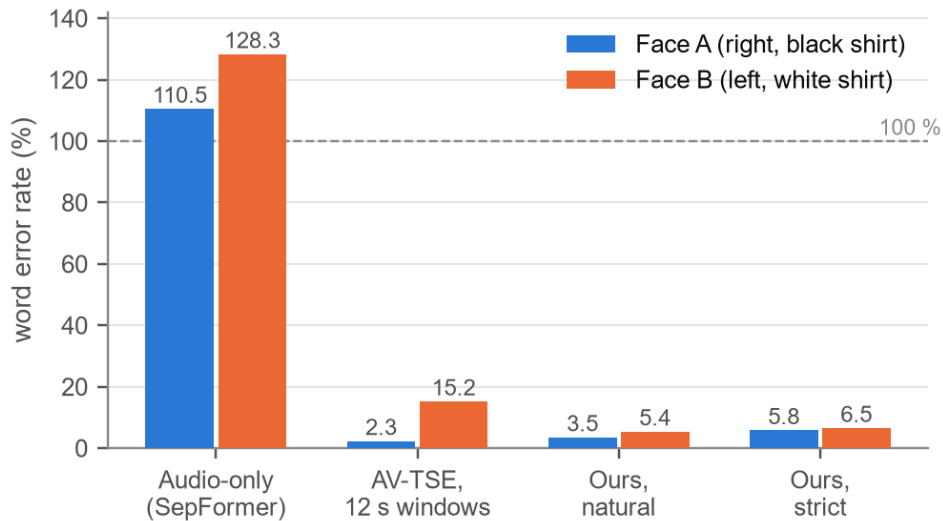


Figure 12: Word error rate on the 37 s phone recording (lower is better).

RQ2 — window length. Shorter windows reduced the similarity between the channels monotonically on both overlapped clips and made no difference on the turn-taking talk show (Figure 2); 2 s windows are used.

RQ3 — leakage. On the bench, refinement and leak cancellation raised the signal-to-interference ratio to 16.3/13.4 dB (Figure 6). On the real videos (Figure 13) strict mode reduced the similarity between the channels from 0.45 to 0.30 and from 0.47 to 0.31 on the two overlapped phone recordings, and left the already distinct news debate (0.08) and talk show (0.13) unchanged: the canceller removes only what is coherent between channels, so it does nothing where there is nothing to remove. The price is clarity: the word error rate on the 37 s clip rose from 3.5/5.4 % to 5.8/6.5 %, and more on the 20 s clip.

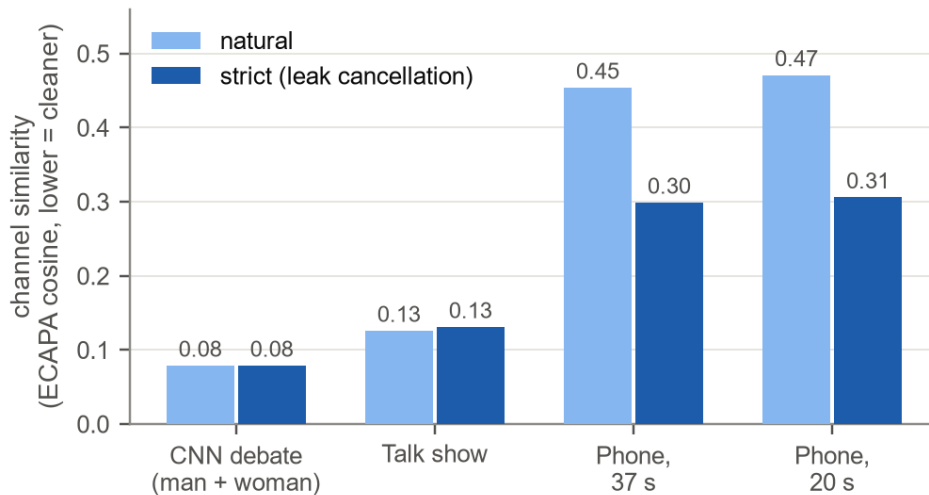


Figure 13: Cross-talk on four videos: similarity between the two isolated channels in natural and strict mode (lower is cleaner).

RQ4 — silence. With the final gate settings 0.05–1.6 % of word time is muted, against 12.2–17.7 % with the V2 settings (Table 7). Table 18 lists the share of output samples that are exactly 0.0 in the final configuration; the share is high where a person is silent or off screen for long periods and low in the phone recordings, where both people read throughout. With the V3 configuration on the talk-show regression clip, a “ghost test” amplified every scored pause by 40 dB: all scored pause samples were bit-exact 0.0 on both channels, and the gate removed only 0.01 % and 0.00 % of speech, against 0.43 % and 3.68 % for the V2 gate (Table 19).

Video	Channel A	Channel B	Channel C
Split-screen news debate	31.9 %	0.6 %	—
Bengali talk show	4.6 %	7.3 %	—
Phone recording, 20 s	1.3 %	0.0 %	—
Phone recording, 37 s	2.4 %	0.1 %	—
Edited talk show (includes off-screen time)	57.2 %	82.5 %	90.6 %

Table 18: Share of output samples that are exactly 0.0 (final configuration).

Version and channel	Scored pause	Bit-exact zero	Speech removed by the gate
V3 (AV-TSE), channel 0	3.59 s in 13 pauses	100.000 %	0.01 %
V3 (AV-TSE), channel 1	2.05 s in 3 pauses	100.000 %	0.00 %
V2 (SepFormer), channel 0	4.83 s in 15 pauses	100.000 %	0.43 %
V2 (SepFormer), channel 1	11.89 s in 43 pauses	100.000 %	3.68 %

Table 19: Ghost test on the talk-show regression clip: pauses amplified by 40 dB.

RQ5 — generality. The correlation map and spectrograms of Section 4.1 show the correct face–voice pairing on the news debate; re-identification raised the tracked share of each person on the edited talk show from 0.2–1.6 % to 44/18/8 %; and processing ran at 1.1–2.5 times real time on all five videos (Table 13).

Rejected alternatives. Besides the approaches in Table 9, we implemented and rejected an ECAPA speaker-identity gate intended to remove voices of people who are never on screen. It could not tell the two on-screen voices of our recordings apart (leave-one-out accuracy 45–53 %, that is, chance) and muted 45–78 % of correct speech, and on the talk show its estimate of the number of speakers changed with the window length. It is therefore disabled by default and available only as a command-line option.

5.3.4 Interpretation and Conclusions on Data

- The lips are the decisive cue. Audio-only separation and lip–audio matching both failed — the latter at the level of chance — while lip-conditioned extraction brought the word error rate down to a few per cent.
- A pretrained model must be operated in its training regime. Short windows were better than long ones, contrary to the intuition that more context always helps.
- Isolation and fidelity trade against each other at every stage. Exposing the strict/natural choice to the listener turned a design conflict that has no single right answer into a user preference.
- Transcription metrics understate leakage; the bench SIR and listening must accompany the word error rate. The residual of 13–16 dB in strict mode for two similar, continuous voices is audible and is the main remaining limitation.
- Distinct voices are much easier than our deliberately hard recording: channel similarity was 0.08–0.13 on the broadcast clips against 0.30 on the phone recordings.
- All numbers come from five clips. They show how the system behaves, not a guaranteed performance on unseen recordings.

5.4 Limitations of Tools (PO(e))

Table 20 lists the technological limits, tolerances and probable errors of the tools used in this project.

Tool	Limit or tolerance	Probable error or consequence
AV-MossFormer2 TSE (16 kHz)	Audio bandwidth 0–8 kHz; lips as 112 × 112 grey images at 25 frames/s; trained on about 3 s windows	“s” and “f” sounds are duller than in the original; with two similar voices overlapping continuously the other voice stays 13–16 dB down (strict) or 8–10 dB down (natural)
Lip video	Face at least about 150 px wide, mostly frontal and visible	Smaller or turned faces extract worse (a 126 px face triggers a warning); a face off screen for more than 0.48 s is muted; a person who is never on screen cannot be isolated and may leak into a visible person’s channel while that person is silent
MediaPipe 0.10.21	Last release with the legacy face API; detection every 5th frame, up to four faces	Pins Python 3.12; fast motion between detections is followed by the tracker only
SFace / YuNet re-identification	Same-identity cosine threshold 0.363	Profile views, low resolution or look-alike people can cause identity errors; tracked share of the edited talk show 8–44 % per person
ECAPA-TDNN identity gate	Needs distinct voices	Chance level (45–53 %) on two similar voices; disabled by default
GPU: RTX 5060 Laptop, 8 GB	1.1–2.5 times real time; uploads of at most 5 min and 512 MB; about 276 + 124 MiB per second of window	Not real-time; long files take minutes; needs the CUDA 12.8 build of PyTorch (sm_120)
Numerical precision (TF32)	Output differs from FP32 by a component 63 dB down	Inaudible; bf16 (33 dB down) was rejected
Frame-rate conversion (ffmpeg)	25 frames/s working video	Up to 20 ms (half a frame) of audio–video timing error
Browser (Web Audio)	Video rate correction of $\pm 2\%$; re-seek beyond 250 ms; video callbacks pause in a hidden tab	Brief visual drift after switching tabs, corrected on return; audio starts only after a user click
Evaluation tools (Whisper, ECAPA similarity)	Proxies without studio ground truth	Whisper follows the louder voice, so the word error rate hides leakage; transcription errors add a floor; part of one script was reconstructed

Table 20: Limitations, tolerances and probable errors of the tools.

5.5 Impact Assessment (PO(f))

5.5.1 Assessment of Societal and Cultural Issues

The main benefit is access to speech. Listeners who cannot separate overlapping voices — many older and hearing-impaired people, non-native listeners, and anyone reviewing a noisy recording — can follow one person in a meeting, a class, an interview or a televised debate. Because the cue is visual, the tool works in Bengali as well as English, which matters in a country where most speech technology targets English. Journalists and archivists can transcribe overlapping speech more reliably.

The same capability can be misused to listen in on one person in a video recorded without their knowledge, and an isolated voice can be taken out of context. The design limits these risks: it works only on people who are visible, it runs locally rather than as a public service, it creates no speech (it only removes other voices, so it cannot put words in anyone’s mouth), and its limits are documented so that its output is not over-trusted.

5.5.2 Assessment of Health and Safety Issues

The tool can support hearing health as the basis of a hearing-assistance aid, since isolating one voice reduces listening effort. The design avoids acoustic hazards: no gain is added beyond the original level, switching and gating are ramped, and pauses are silent rather than noisy. There is no electrical or mechanical hazard beyond the ordinary use of a laptop. The system is not a medical device and has not been validated for clinical use; users should listen at moderate volume.

5.5.3 Assessment of Legal Issues

Consent and privacy. Recording and processing people’s voices is subject to privacy and data-protection law. Our showcase recordings are of team members who consented; broadcast clips were used only for internal testing, were not redistributed, and appear in this report only as screenshots for academic evaluation. Because processing is local, no personal data is transferred to third parties.

Copyright and licences. The AV-MossFormer2 code is Apache-2.0 licensed (its visual front-end is MIT licensed), and the vendored copy keeps the original notices and a record of its source. The model card of the weights does not document the training data; related checkpoints are trained on audio-visual corpora such as LRS2 and VoxCeleb2 [32] whose terms restrict use to research, so we treat the weights as suitable for non-commercial use only. SpeechBrain, MediaPipe and OpenCV are Apache-2.0 licensed and Whisper is MIT licensed. A commercial product would need weights trained on data that permits commercial use.

5.6 Sustainability Evaluation (PO(g))

Energy. On the RTX 5060 Laptop GPU, with a power limit of about 91 W and a real-time factor of about 2, the energy per minute of processed video is

$$E_{\min} = P \cdot \text{RTF} \cdot 60 \text{ s} \approx 91 \text{ W} \times 2 \times 60 \text{ s} \approx 10.9 \text{ kJ} \approx 3.0 \text{ Wh} \quad (21)$$

so one hour of video needs about 0.18 kWh — roughly BDT 1.8 of electricity at an assumed tariff of BDT 10 per kWh, and about 0.1 kg of CO₂ at an assumed grid intensity of 0.6 kg CO₂ per kWh. Playback afterwards is almost free, because it only mixes precomputed channels.

Environmental sustainability. No model was trained; the system reuses open pretrained models, skips off-screen windows (−52 % computation on edited video) and uses TF32 arithmetic (−17 % time). It runs on hardware the team already owned, and local processing avoids transferring video to a data centre.

Technical sustainability. Vendoring the model code reduced the project to a single Python environment. Versions are pinned to the newest matched PyTorch/torchaudio pair (2.11); a preflight script and a GPU probe check a new machine before it is used; every tunable number lives in one file together with the measurement that justifies it; and a re-vendoring script re-applies our two edits to future versions of the model code. The main long-term risks are that torchaudio is in maintenance mode and that MediaPipe’s legacy face API pins Python 3.12; audio input and output already avoid torchaudio.

Social and economic sustainability. The software adds no cost for its users, runs on common gaming laptops, and is shared openly, so it can be maintained and extended by others.

5.7 Ethical Issues (PO(h))

We applied the following ethical principles while designing and evaluating the project.

- **Consent.** The showcase recordings are of team members, who agreed to be recorded and to have the recordings shown in the presentation and in this report.
- **Privacy.** All processing stays on the user's machine; nothing is uploaded or stored elsewhere, and job files remain in the project folder, where the user can delete them.
- **Preventing misuse.** The system needs a visible face, creates channels only for people on screen, and is not designed for covert surveillance.
- **Honest reporting.** We report the residual leakage (13–16 dB), our failures (the matcher at chance, the identity gate) and the methods we rejected, with numbers. Objective scores are computed on ungated output, and definitions were not changed to make results look better: when a leakage statistic in the metadata turned out to be meaningless, it was left unchanged and a correct test was added rather than redefining the published field.
- **Attribution.** Vendored code keeps its Apache-2.0 and MIT notices and a record of its origin; every model, dataset and tool is cited.
- **Use of AI tools.** AI coding assistants were used for parts of the implementation, the diagnosis and the drafting of documentation, including this report. Every result was reproduced and checked by the team, which takes responsibility for the content.

The main ethical challenge was the use of broadcast clips, which show people who did not consent to our experiment. We mitigated it by using them only for internal testing, not redistributing them, and showing only screenshots needed for evaluation.

Ethics declaration statement. We, the members of Group 04, Section C2, declare that this project was carried out in accordance with the ethical principles stated above. The recordings of people used in this work were made with their consent or taken from public broadcasts for internal testing only; no personal data was shared with third parties; all external code, models and data are acknowledged; the results reported here are our own measurements, reported without selective omission; and the use of AI tools is disclosed. We accept responsibility for the content of this report.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Member	Technical contribution	Presentation
Redwan Nur Alam Shilad 2206177	Literature review and problem scope; test-video set and success criteria; Introduction and Problem Formulation (PO(b)) sections	Slides 1–8: problem
Ahnaf Hasan Adeeb 2206178	System architecture and AV-TSE integration; DSP stages (refinement, leak canceller, gate); FastAPI backend and RTX 5060 set-up	Slides 9–14: design and demonstration
S. M. Khairul Islam Rifat 2206179	Ground-truth leak bench and word-error-rate scoring; model comparisons (SepFormer, Dolphin); re-identification tests on edited video	Slides 15–22: evaluation
Md. Shahriare Arefin Shuvo 2206180	Vision set-up (MediaPipe coordinate fix, face tracking); Web Audio player and user interface; ethics, cost analysis and documentation	Slides 23–31: wrap-up

Table 21: Individual contributions.

6.2 Mode of TeamWork

The team worked from one shared design document — first the V2 architecture, then V3 — which recorded every decision together with the measurement that justified it, so there was a single source of truth that everyone could consult. We followed a “measure first” rule: a claim entered the design only after a diagnostic script had tested it, which produced the 45 diagnostic and verification scripts in the repository. Regression clips were re-run after every change, so a result counted only when it was reproduced.

Work was divided by subsystem — problem framing, DSP and backend, evaluation, and vision and player — with clear interfaces between the parts (the separator interface and the job output format), so members could work in parallel. Two members mainly built and two mainly framed and measured, so every claim was checked by someone other than its author. For the final presentation each member presented one quarter, and each can explain the whole system.

6.3 Diversity Statement of Team

The team combines complementary skills: signal-processing theory, deployment of deep-learning models, web engineering and experimental evaluation. It also combines working styles: builders who implement and checkers who frame the problem and test claims, which kept the project honest. Ownership was shared, and every member presented part of the work. We also aimed for inclusive testing, covering English and Bengali, male and female voices, and phone and broadcast video, so that the system would serve more than one kind of user.

6.4 Log Book of Project Implementation

Date	Milestone achieved	Individual Role	Team Role	Comments
Before 10 Aug 2026	Colab prototype: SepFormer separation with lip-loudness matching	Ahnaf: separation pipeline; Shuvo: face and lip tracking	Chose the problem; collected the first test clips	“Ghost whisper” and speaker swaps heard

Date	Milestone achieved	Individual Role	Team Role	Comments
10–11 Aug	V2 design and rebuild: output-level fix, Wiener mask, Schmitt gate, FastAPI server and Web Audio player	Ahnaf: DSP chain and backend; Shuvo: Web Audio player	Reviewed the V2 design document	Ghost whisper traced to peak normalisation
12 Aug	Matcher diagnosis: lip–loudness matching no better than white noise	Rifat: diagnostic measurements; Redwan: literature on audio-visual methods	Decided to replace matching	0.0679 vs 0.0683
13 Aug – 6 Sep	V2 tested on more than 16 real videos; gate re-fitted; face-coordinate fix	Rifat: testing and metrics; Shuvo: MediaPipe fix and tracking; Redwan: test-video set	Prepared the progress update	5 of 7 planned milestones done
7 Sep	Progress update presented	Each member presented part of the slides	Listed five open challenges	See Table 14
8–18 Sep	Literature study of audio-visual extraction; V3 design	Redwan: literature review; Ahnaf: V3 architecture	Chose AV-MossFormer2 and vendoring	One Python environment
19–23 Sep	V3: AV-MossFormer2 integrated; mouth regions; ghost test	Ahnaf: integration; Rifat: ghost test and verification	Reviewed results on the regression clip	Scored pauses 100 % bit-exact silent
24 Sep	RTX 5060 set-up; showcase recording; 2 s windows, refinement, leak cancellation, re-identification, strict/natural player	Ahnaf: DSP and set-up; Rifat: leak bench, WER, Dolphin; Shuvo: player; Redwan: test set	Two members recorded the showcase clip	WER 3.5/5.4 %; SIR 16.3/13.4 dB
24–25 Sep	Final slides, demonstration video and report	Each member: one quarter of the slides and the related sections	Joint review	–

Table 22: Log book of the project (dates from the project’s files and design documents).

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

BUET students build a “click to listen” tool for crowded conversations

Dhaka, September 2026. Four students of the Department of EEE, BUET, have built software that picks one voice out of a video in which people talk over each other. The user uploads a video and clicks a face; the program watches that person’s lips and plays only their voice, and switching to another person is instant. On the team’s own recording of two people reading different texts at once, the isolated voices were transcribed with only 3.5–5.4 % word errors, against more than 100 % for methods that use sound alone. The system works in Bengali and English, runs on an ordinary gaming laptop, and never uploads the video anywhere.

7.2 User Manual

System requirements. Windows 10 or 11; an NVIDIA GPU with at least 4 GB of memory (8 GB recommended); Python 3.12; ffmpeg; about 5 GB of free disk space; a current Chrome, Edge or Firefox browser. An internet connection is needed only once, to download the model weights.

Installation

1. Copy the project folder to the computer and open a terminal in it.
2. Create the Python environment: `py -3.12 -m venv .venv`, then activate it with `.venv\Scripts\activate`.
3. Install PyTorch for the GPU: `pip install torch==2.11.0 torchaudio==2.11.0 --index-url`

<https://download.pytorch.org/whl/cu128> (use cu126 instead of cu128 for GTX 10-series cards).

4. Install the remaining packages: `pip install -r requirements.txt`.
5. Install ffmpeg, for example with `winget install Gyan.FFmpeg`, and check it with `ffmpeg -version`.
6. Check the machine: run `.venv\Scripts\python.exe scripts\gpu_probe.py`, then `run.bat --preflight`. The preflight checks Python, NumPy, CUDA, ffmpeg and the silence gate, and downloads the model weights once.

Using the application

1. Start the server with `run.bat` and open <http://127.0.0.1:8000> in the browser.
2. Drag a video onto the page, or click “browse” (mp4, mov, mkv, webm, avi or m4v; at most 5 minutes and 512 MB).
3. Wait while the progress bar shows each processing stage; this takes about one to two and a half times the length of the video on an RTX 5060.
4. Press Play (or the Space bar). A box is drawn around every detected face.
5. Click a face, or its “Speaker” button, to hear only that person; click another face to switch instantly.
6. Click “Isolation: strict” to change to natural isolation (a clearer voice with slightly more of the other speaker), and click again to return.
7. Click “New video” (or press N) to process another file.

Keyboard shortcuts: Space — play or pause; ← and → — seek 5 s; 1–9 — select a speaker; N — new video.

Command line (no browser): `python -m app.pipeline input.mp4 --out runs/cli` writes the same outputs. Add `--identity-gate` for a clip with an off-screen speaker, or `--separator sepformer` for audio-only separation.

Symptom	Likely cause	Remedy
“No kernel image is available” or another CUDA error	The installed PyTorch build does not support the GPU	Reinstall PyTorch from the matching CUDA index (step 3) and run <code>gpu_probe.py</code>
Processing is very slow	The CPU-only PyTorch was installed	Install PyTorch from the CUDA index (step 3)
No sound after pressing Play	The browser blocks audio until the page is clicked	Click Play again
A person sounds wrong or has no button	Face too small (under about 150 px), turned away or off screen	Use a higher-resolution video in which faces are visible
The page cannot be reached	The server is not running, or an old server holds the port	Restart <code>run.bat</code> ; set the PORT variable to use another port
“File exceeds 512 MB”	Upload limit	Trim or compress the video (at most 5 minutes)

Table 23: Troubleshooting.

7.3 Github Link

[Add the GitHub repository link here] — the repository contains the source code, a README with the set-up steps, the design documents (V2 and V3), the diagnostic scripts and ready-to-play demonstration jobs.

7.4 YouTube Link

[Add the YouTube link of the demonstration video here] — video title: “Audio-Visual Speaker Isolation | C2.4 | EEE 312 (2026) Final Project | Dept of EEE, BUET”.

8 Project Management and Cost Analysis (PO(k))

8.1 Bill of Materials

Item	Specification	Qty	Cost to project (BDT)	Remark
Laptop computer	NVIDIA RTX 5060 Laptop GPU, 8 GB	1	0	Already owned by a team member
Development computer	NVIDIA GTX 1050 Ti, 4 GB	1	0	Already owned; used for V2 and V3
Smartphone camera	iPhone, 4K 60 frames/s video, 48 kHz audio	1	0	Already owned; showcase recording
Software	Python, PyTorch, FastAPI, MediaPipe, OpenCV, SpeechBrain, ffmpeg	—	0	Open source
Pretrained models	AV-MossFormer2 TSE, SFace, YuNet, ECAPA-TDNN, Whisper (evaluation)	—	0	Open weights; non-commercial use
Electricity	About 3 Wh per minute of processed video	—	≈ 0.03 per minute of video	At an assumed BDT 10 per kWh
Total extra cost			0 (plus negligible electricity)	

Table 24: Bill of materials.

8.2 Calculation of Per Unit Cost of Prototype

Because all hardware was already owned and all software is free, the incremental cost of the prototype is zero. Its operating cost is the electricity used for processing. With the energy of Eq. (21) and an assumed tariff c_e of BDT 10 per kWh,

$$C_{\min} = E_{\min} c_e \approx 0.0030 \text{ kWh} \times 10 \text{ BDT/kWh} \approx 0.03 \text{ BDT} \quad (22)$$

per minute of video, or about BDT 1.8 per hour of video. If the hardware had to be bought, the laptop would dominate the cost; any computer with an NVIDIA GPU of at least 4 GB can run the software, more slowly.

8.3 Calculation of Per Unit Cost of Mass-Produced Unit

The product is software, so an additional unit is a copy: distribution through GitHub costs nothing, and the cost per unit is the user’s electricity (about BDT 0.03 per minute of video) on hardware they already own. Two costs would appear in a commercial product: licensing, or retraining, the model on data that permits commercial use, since the current weights are for research use; and support and maintenance. A hosted service would add GPU server costs and would also require users to upload their videos, which conflicts with our privacy design, so local deployment is recommended.

8.4 Timeline of Project Implementation

Figure 14 shows the timeline of the project, reconstructed from the dates of our design documents and files. The project followed an incremental build order in which every step ended with a working demonstration, so there was always a system that ran.

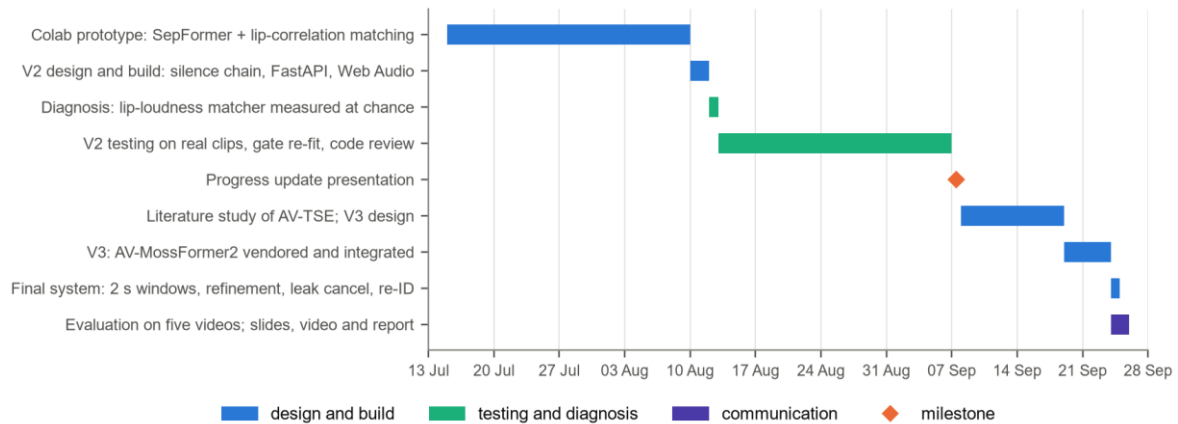


Figure 14: Gantt chart of the project (2026).

9 Future Work (PO(I))

1. **Ground truth.** Record solo takes of each speaker, mix them, and score SI-SDR [33], PESQ [34] and STOI [35] on the ungated output, so that fidelity is measured against a true reference.
2. **Harder overlap.** Fine-tune the extraction model on similar voices that overlap continuously, the case in which 13–16 dB of leakage remains.
3. **Full bandwidth.** Restore the band above 8 kHz, for example by extending the bandwidth of the isolated voice with guidance from the original 48 kHz recording.
4. **Off-screen voices.** Enrol each voice while the person is visible and follow it when they are not, combining lip conditioning with speaker-conditioned extraction [10].
5. **Real time.** Move to streaming inference with a short look-ahead, for live calls and hearing assistance.

The project required learning far beyond the course: reading current research papers, understanding pretrained models from their source code, and verifying every claim by measurement. We intend to keep these habits — following the literature, building measurement benches before trusting a result, and documenting negative results — in future projects and in our careers.

10 References

- [1] E. C. Cherry, “Some experiments on the recognition of speech, with one and with two ears,” *J. Acoust. Soc. Am.*, vol. 25, no. 5, pp. 975–979, 1953.
- [2] A. Hyvärinen and E. Oja, “Independent component analysis: Algorithms and applications,” *Neural Networks*, vol. 13, no. 4–5, pp. 411–430, 2000.
- [3] J. R. Hershey, Z. Chen, J. Le Roux, and S. Watanabe, “Deep clustering: Discriminative embeddings for segmentation and separation,” in *Proc. IEEE ICASSP*, 2016, pp. 31–35.
- [4] D. Yu, M. Kolbæk, Z.-H. Tan, and J. Jensen, “Permutation invariant training of deep models for speaker-independent multi-talker speech separation,” in *Proc. IEEE ICASSP*, 2017, pp. 241–245.
- [5] Y. Luo and N. Mesgarani, “Conv-TasNet: Surpassing ideal time–frequency magnitude masking for

- speech separation,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 27, no. 8, pp. 1256–1266, 2019.
- [6] C. Subakan, M. Ravanelli, S. Cornell, M. Bronzi, and J. Zhong, “Attention is all you need in speech separation,” in *Proc. IEEE ICASSP*, 2021, pp. 21–25.
 - [7] S. Zhao *et al.*, “MossFormer2: Combining transformer and RNN-free recurrent network for enhanced time-domain monaural speech separation,” in *Proc. IEEE ICASSP*, 2024.
 - [8] M. Maciejewski, G. Wichern, E. McQuinn, and J. Le Roux, “WHAMR!: Noisy and reverberant single-channel speech separation,” in *Proc. IEEE ICASSP*, 2020, pp. 696–700.
 - [9] K. Žmolíková *et al.*, “SpeakerBeam: Speaker aware neural network for target speaker extraction in speech mixtures,” *IEEE J. Sel. Topics Signal Process.*, vol. 13, no. 4, pp. 800–814, 2019.
 - [10] Q. Wang *et al.*, “VoiceFilter: Targeted voice separation by speaker-conditioned spectrogram masking,” in *Proc. Interspeech*, 2019, pp. 2728–2732.
 - [11] A. Ephrat *et al.*, “Looking to listen at the cocktail party: A speaker-independent audio-visual model for speech separation,” *ACM Trans. Graph.*, vol. 37, no. 4, Art. 112, 2018.
 - [12] T. Afouras, J. S. Chung, and A. Zisserman, “The conversation: Deep audio-visual speech enhancement,” in *Proc. Interspeech*, 2018, pp. 3244–3248.
 - [13] R. Gao and K. Grauman, “VisualVoice: Audio-visual speech separation with cross-modal consistency,” in *Proc. IEEE/CVF CVPR*, 2021.
 - [14] ModelScope (Alibaba Tongyi Lab), “ClearerVoice-Studio: AV-MossFormer2 target speaker extraction, checkpoint alibabasglab/AV_MossFormer2_TSE_16K,” GitHub repository. [Online]. Available: <https://github.com/modelscope/ClearerVoice-Studio> (accessed Sep. 19, 2026).
 - [15] K. Li *et al.*, “Efficient audio-visual speech separation with discrete lip semantics and multi-scale global-local attention,” in *Proc. ICLR*, 2026, arXiv:2509.23610.
 - [16] C. Lugaresi *et al.*, “MediaPipe: A framework for building perception pipelines,” arXiv:1906.08172, 2019.
 - [17] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li, and D. Wen, “SFace: Sigmoid-constrained hypersphere loss for robust face recognition,” *IEEE Trans. Image Process.*, vol. 30, pp. 2587–2598, 2021.
 - [18] W. Wu, H. Peng, and S. Yu, “YuNet: A tiny millisecond-level face detector,” *Machine Intelligence Research*, vol. 20, no. 5, pp. 656–665, 2023.
 - [19] B. Desplanques, J. Thienpondt, and K. Demuynck, “ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification,” in *Proc. Interspeech*, 2020, pp. 3830–3834.
 - [20] M. Ravanelli *et al.*, “SpeechBrain: A general-purpose speech toolkit,” arXiv:2106.04624, 2021.
 - [21] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, 2023, pp. 28492–28518.
 - [22] J. B. Allen, “Short term spectral analysis, synthesis, and modification by discrete Fourier transform,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 25, no. 3, pp. 235–238, 1977.
 - [23] D. W. Griffin and J. S. Lim, “Signal estimation from modified short-time Fourier transform,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 32, no. 2, pp. 236–243, 1984.
 - [24] N. Wiener, *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. Cambridge, MA, USA: MIT Press, 1949.
 - [25] A. V. Oppenheim, R. W. Schaffer, and J. R. Buck, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.
 - [26] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 5th ed. Pearson, 2022.
 - [27] E. Vincent, R. Gribonval, and C. Févotte, “Performance measurement in blind audio source separation,” *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 14, no. 4, pp. 1462–1469, 2006.
 - [28] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Res. Logist. Q.*, vol. 2, no. 1–2, pp. 83–97, 1955.
 - [29] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. NeurIPS*, 2019, pp. 8024–8035.
 - [30] W3C, “Web Audio API,” W3C Recommendation, Jun. 2021. [Online]. Available: <https://www.w3.org/TR/webaudio/>
 - [31] S. Wisdom *et al.*, “Differentiable consistency constraints for improved deep speech enhancement,” in

Proc. IEEE ICASSP, 2019, pp. 900–904.

- [32] J. S. Chung, A. Nagrani, and A. Zisserman, “VoxCeleb2: Deep speaker recognition,” in *Proc. Interspeech*, 2018, pp. 1086–1090.
- [33] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, “SDR – half-baked or well done?” in *Proc. IEEE ICASSP*, 2019, pp. 626–630.
- [34] ITU-T, “Perceptual evaluation of speech quality (PESQ): An objective method for end-to-end speech quality assessment of narrow-band telephone networks and speech codecs,” Recommendation P.862, 2001.
- [35] C. H. Taal, R. C. Hendriks, R. Heusdens, and J. Jensen, “An algorithm for intelligibility prediction of time–frequency weighted noisy speech,” *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 19, no. 7, pp. 2125–2136, 2011.